

ASP.NET CORE CON C#

1. Introducción

La plataforma .NET ofrece una serie de herramientas y tecnologías necesarias para construir y desarrollar aplicaciones Web del lado del servidor. Las aplicaciones ASP.NET Core se van a desarrollar dentro del entorno de ejecución que nos facilita el .NET Core. ASP.NET es una parte de la plataforma .NET. Fue, en su momento, la parte más innovadora.

ASP.NET Core son las aplicaciones web desarrolladas con las nuevas versiones del framework (Core)

Comprende varias tecnologías:

a) **Sitios Web** para la creación de aplicaciones de Internet con interfaces de usuario a las que se puede acceder mediante un explorador. Son las herederas de las antiguas ASP (Active Server Pages). Podemos crear varios tipos de proyectos:

1. **WebForms**: equivalentes a los WindowsForms que hemos visto (WinForm)

La filosofía de ASP.NET es sencilla: las páginas ASP.NET, también llamadas páginas activas, son páginas que contienen código HTML, script de cliente y un código que se ejecuta en el servidor, dando como resultado código HTML. Son páginas que se ejecutan en el servidor enviando como resultado al cliente código HTML. ASP.NET soporta diversos lenguajes: C#, Visual Basic, JScript. Se ejecutan sobre el CLR (Common Language Runtime)

Ventajas:

- o Los Web Forms contienen controles de servidor, controles como el GridView o el ListView, contienen funcionalidad para enlazar datos.
- o ViewState: Normalmente los controles no retienen los valores entre los pedidos al servidor, pero en los Web Forms esto se logra guardando el último estado conocido en un campo oculto denominado ViewState.
- o Programación manejada por eventos
- o Desarrollo de aplicaciones rápido: Con los controles de servidor + el modelo manejado por eventos + ViewState.

- o Curva de aprendizaje pequeña: Es posible hacer aplicaciones con muy poco conocimiento de HTML y JavaScript.

Desventajas:

- o El Code Behind (archivo que contiene código a ejecutarse en el servidor) termina conteniendo muchos manejadores de eventos
- o Performance: ViewState se convirtió en una solución para varios problemas de ASP, pero como se guarda en la misma página hace que el tamaño de la misma se incremente
- o Reutilización: El código incluido en el Code Behind de un Web Form no es fácilmente reutilizable en otro.
- o Poco control sobre el HTML: Muchas veces no está muy claro cuál es el HTML que resultará de la utilización de los controles, lo cual hace complicada la integración con frameworks como jQuery.
- o SEO: No se generan URLs amigables, lo cual afecta el SEO de la página.
- o Trabajo en paralelo: Como cada Web Form está atado a su Code Behind, no es recomendable que dos desarrolladores trabajen al mismo tiempo en estos archivos.

2. ASP.NET CORE

2.1 MVC: modelo vista controlador. Separación entre las distintas capas lógica de negocio, los datos y la presentación

Ventajas:

- o Arquitectura de proyecto: Al estar fuertemente implementada la separación de responsabilidades, también tenemos una arquitectura del proyecto ordenada.
- o Los controles son clases separadas, por lo cual, es posible hacer test unitarios
- o Reutilización: Los controles no están atados a una vista, por lo cual pueden ser reutilizados.
- o Las páginas son mucho más livianas comparadas con los Web Forms.
- o Control total del HTML: Como no existen los controles de servidor, la única opción es utilizar los controles HTML. La integración con librerías JavaScript es más simple.
- o Al estar todo separado, es posible que un desarrollador esté trabajando en una vista, mientras otro esté en el controlador y un tercero esté en el modelo, sin que interfieran entre ellos.

- o Soporte para múltiples motores de vistas como aspx, razor, etc.
- o Características preexistentes de ASP.NET: Como está construido sobre el framework ASP.NET Core, provee características como autenticación, session, etc.

2.2 Razor Pages : lo que se puede hacer en MVC se puede hacer en Razor. En lugar de tener un controlador, tienes varios

2.3 Web APIs : la aplicación es sólo el backend . Es decir no tenemos vistas y sólo tenemos modelos y controladores.

Por ejemplo, tenemos una aplicación con varios fronted (angular para navegadores, android para móviles). Queremos visualizar nuestra factura, la web api genera en el back esa factura que será enviada a cada front. Cada front recibe los datos y genera la página que verá el usuario.

Desventajas:

- o La principal desventaja de ASP.NET MVC es que presenta una curva de aprendizaje mucho más importante.
- b) **MicroServicios Web** para la creación de aplicaciones no navegables a las que se accede en forma de programa. Pequeñas aplicaciones que se ejecutan independientemente pero que conforman una sola.